

DHANU SHREE V

9952023889

dhanushreevaideyanathan@gmail.com

-SUMMARY

- Results-driven Frontend Developer with 3+ years of experience building responsive, scalable web applications using React.js, Next.js, TypeScript, Redux, Node.js, and Express.js across E-Commerce, SaaS, EdTech, FinTech, and Healthcare domains.
- Implemented React 18 concurrent features using useTransition for non-blocking SaaS dashboard search and useDeferredValue for live query suggestion rendering maintaining smooth 60fps UI on low-powered devices.
- Built custom React Hooks library (useDebounce, useIntersectionObserver, useWebSocket, useLocalStorageState) published to internal npm package consumed across 5 product modules reducing boilerplate by 30%.
- Architected Redux Toolkit with RTK Query replacing manual thunk patterns enabling auto-caching, optimistic mutations, and background refetching for real-time dashboards.
- Developed Next.js App Router applications with parallel routes and intercepting routes enabling modal-based detail overlays with deep-link support and persistent URL state.
- Built scalable backend services using Node.js and Express.js implementing REST APIs, microservices architecture, and real-time communication systems using WebSocket.
- Integrated GraphQL (Basic) with Apollo Client and implemented Kafka-based event streaming for asynchronous data processing and scalable distributed workflows.
- Applied Python with NumPy, Pandas, and OpenCV for data processing utilities, analytics scripts, and basic image processing automation use cases.
- Achieved 30–35% reduction in page load times using next/image optimization, code splitting, memoization, and critical CSS strategies.
- Implemented WCAG 2.1 accessibility compliance ensuring inclusive UI/UX across enterprise-grade applications.
- Configured CI/CD pipelines using GitHub Actions with automated testing, linting, and deployment workflows integrated with cloud platforms.

SKILLS

Frontend Frameworks & Libraries React.js, Next.js (App

Router, SSR/SSG/ISR), Redux Toolkit, RTK Query, Redux Thunk, Redux Saga, React Hooks (custom), Context API, React 18 (useTransition, useDeferredValue)

Backend Node.js, Express.js, REST APIs, Microservices, WebSocket, GraphQL (Basic), Kafka, Python, OpenCV, NumPy, Pandas

Programming Languages JavaScript (ES6+), TypeScript, HTML5, CSS3

Styling & UI Tailwind CSS, Bootstrap, Material UI, SASS/SCSS, Styled Components, Responsive Design, WCAG 2.1

Performance & Architecture Code Splitting, Lazy

Testing & Debugging Jest, Cypress, React Testing Library,

Loading, Memoization, ISR/SSR/SSG, Critical CSS

Inlining, next/image Optimisation, Virtual DOM **API &**

Real-Time REST APIs, GraphQL (Apollo Client),

WebSocket, JWT, OAuth 2.0, RTK Query, Axios, Fetch

MSW (Mock Service Worker), Storybook, Chrome

DevTools, Lighthouse, Postman

CI/CD & DevOps GitHub Actions, Jenkins, Git, GitHub,

GitLab, Bitbucket, Webpack, Vite

Security & Compliance RBAC, MFA, HTTPS, CORS,

WCAG 2.1 Accessibility, Cross-Browser Compatibility

Deployment AWS S3, CloudFront, AWS Amplify,

Firebase Hosting, Vercel

-EXPERIENCE

TATA CONSULTANCY SERVICES (TCS)

Assistant System Engineer 08/2022 to 04/2026

-EDUCATION AND TRAINING

2018 - 2022

Bannari Amman Institute of Technology , B.E. Electrical And Electronics Engineering

PROJECT

PROJECT 1: CartNest – Shopmatic Multi-Channel Commerce | Domain: E-Commerce / SaaS

Client: Shopmatic (Singapore-based SMB e-commerce SaaS) | Domain: E-Commerce / Multi-Channel
Retail Tech Stack: React.js, Redux Toolkit, RTK Query, TypeScript, Tailwind CSS, Node.js, Express.js, REST APIs, Microservices, WebSocket, GraphQL (Basic), Kafka, JWT, AWS S3, GitHub Actions, Jest, React Testing Library, Python (Basics), NumPy, Pandas

Description: Multi-channel e-commerce SaaS platform enabling 25,000+ Southeast Asian merchants to manage storefronts, sync product catalogue across channels, process orders, and track inventory in real time.

Roles & Responsibilities:

- Implemented Redux Toolkit RTK Query replacing 8 manual thunk actions with automatic cache invalidation: mutating a product invalidates both ProductList and InventoryDashboard cache tags triggering simultaneous UI refresh.
- Built custom useInventorySync hook polling RTK Query every 30 seconds for real-time stock level updates preventing oversell on active merchant storefronts.
- Developed dynamic product filtering engine using URL-driven state with React Router v6 supporting 25+ filter dimensions while maintaining deep-link shareability.
- Designed and integrated Node.js + Express.js backend services exposing REST APIs for product, inventory, and order management workflows.
- Built Microservices Architecture separating catalog service, inventory service, and order processing service enabling independent scaling of modules.
- Implemented WebSocket-based real-time inventory updates and order status tracking for live merchant dashboard synchronization.
- Integrated Kafka event streaming for order processing pipeline ensuring asynchronous communication between payment, inventory, and notification services.
- Developed GraphQL (basic) endpoints for flexible product querying across multi-channel storefronts reducing over-fetching in catalog screens.
- Implemented Python-based utility scripts using NumPy and Pandas for inventory analytics and sales trend preprocessing for internal reporting dashboards.
- Implemented JWT-based authentication with role-based route protection (merchant, admin, support_agent).
- Developed responsive mobile-first storefront UI using Tailwind CSS JIT with Shopmatic brand tokens.
- Achieved Lighthouse performance score of 91 through code splitting, lazy loading, and optimized rendering strategies.
- Configured GitHub Actions with Lighthouse CI performance budget checks ensuring production-grade performance consistency.

PROJECT 2: SupportDesk – Freshworks CRM Dashboard | Domain: SaaS / Customer Support

Client: Freshworks (India-based SaaS CRM platform) | Domain: SaaS / Customer Experience Tech
Stack: React.js, Next.js, Redux Saga, TypeScript, Material UI, Node.js, Express.js, REST APIs, GraphQL (Basic), WebSocket, Kafka, GitHub Actions, Jest, Cypress

Description: Real-time customer support and CRM management dashboard enabling 500+ support agents to manage ticket lifecycle, monitor SLA compliance, and track customer interactions across channels.

Roles & Responsibilities:

- Built Next.js App Router dashboard with parallel routes and deep-linked ticket views for seamless agent navigation.
- Implemented Redux Saga WebSocket channel consuming real-time ticket events (new_ticket, status_changed, SLA_breach) and dispatching normalized Redux actions.
- Integrated Apollo Client GraphQL caching layer with normalized ticket entity storage reducing redundant API calls.
- Designed and implemented Node.js + Express.js REST APIs for ticket management, SLA tracking, and agent assignment workflows.
- Built Microservices-based backend architecture separating ticket service, notification service, and analytics service for scalability.
- Implemented WebSocket real-time communication layer enabling instant ticket updates across multiple agent dashboards.
- Integrated Kafka event streaming for SLA monitoring, escalation workflows, and cross-service event propagation.
- Developed GraphQL (basic) APIs for ticket analytics and dashboard aggregation queries improving frontend flexibility.
- Built CSAT analytics dashboard with real-time WebSocket updates for performance metrics visualization.
- Applied Python (NumPy, Pandas) for backend analytics processing of ticket trends, SLA breach prediction, and agent performance metrics.
- Implemented RBAC component guards restricting admin-only actions such as bulk reassignment and priority escalation.
- Optimized performance using virtualized lists handling 10,000+ tickets at 60fps using react-window.
- Configured CI/CD pipelines with GitHub Actions integrating Jest and Cypress tests with automated quality gates.

PROJECT 3: CourseHub – Learnosity Learning Platform | Domain: EdTech / LMS

Client: Learnosity (Ireland-based EdTech SaaS) | **Domain:** EdTech / Digital Learning

Tech Stack: React.js, Context API, Firebase (Auth, Firestore), SASS, React Testing Library, Jest, Firebase Hosting

Description: Collaborative learning management platform enabling 300+ education publishers to deliver adaptive courses, track student progress, manage assessments, and analyse learning outcomes for 30,000+ enrolled learners.

Roles & Responsibilities:

- Designed Context API architecture with 4 separate context providers (AuthContext, CourseContext, AssessmentContext, ProgressContext) preventing prop-drilling across deeply nested course player component tree.
- Built interactive course module player with sequential lesson navigation, completion tracking persisted to Firestore on every lesson finish, and resume-from-last-position on re-entry using stored lesson_progress document.
- Integrated Firebase Authentication with Google Workspace SSO supporting institutional sign-in for 300+ publisher accounts with custom Firestore claims encoding publisher_id, subscription_tier, and content_access_level.
- Implemented Firestore real-time listener in course discussion board updating comment threads, upvote counts, and instructor reply badges without manual refresh enabling collaborative learning discussions in real time.
- Developed adaptive assessment interface displaying next question based on student performance score: correct answer advances to harder question, incorrect triggers easier follow-up using client-side difficulty algorithm.
- Applied React.lazy import() for course video player, PDF viewer, and interactive quiz components loading them on-demand reducing initial assessment page bundle from 1.8MB to 420KB improving student load times.
- Built

progress analytics dashboard for educators using Chart.js rendering completion rate trends, at-risk student identification (< 60% engagement), and competency score breakdown per learning objective per cohort. • Implemented Jest and React Testing Library tests for course navigation, assessment submission, and progress tracking components with Firebase Emulator Suite enabling reliable offline testing of Firestore interactions. • Configured Firebase Hosting multi-site with learner portal and educator dashboard as separate sites sharing Firestore backend with role-based security rules preventing cross-role data access at database level. • Improved UI performance by 25% optimising Firestore listener usage: replaced per-component real-time listeners with centralised subscription management in CourseContext avoiding redundant network connection overhead.

PROJECT 4: WealthView – Moneybox Financial Analytics | Domain: FinTech / Personal Finance

Client: Moneybox (UK-based savings and investment SaaS) | **Domain:** FinTech / Personal Finance

Tech Stack: React.js, Redux Thunk, Chart.js, Tailwind CSS, REST APIs, GitHub Actions, Jest, Storybook

Description: Personal investment and portfolio analytics dashboard enabling 900,000+ Moneybox users to visualise portfolio performance, track ISA contributions, monitor account balances, and analyse investment returns. **Roles &**

Responsibilities:

- Built real-time portfolio value ticker component using Redux Thunk polling Moneybox REST API every 60 seconds dispatching price_updated action updating 200+ holding value cells without unnecessary full dashboard re-render.
- Developed Chart.js custom plugin registry: portfolio performance waterfall chart, holdings pie with explode-on hover, ISA contribution progress gauge, and 90-day price history sparklines per holding card. • Applied React.memo, useMemo, and useCallback preventing unnecessary re-renders of portfolio holding cards during frequent polling updates containing 200+ entries with complex calculated fields per visible row. • Built configurable scenario calculator with sliding range inputs (contribution amount, projected growth rate) updating Chart.js area chart projections using useDeferredValue deferring expensive recalculation on each slider drag.
- Implemented role-based dashboard views: ISA account view hides LISA withdrawal penalty warning for non LISA holders, Pension view shows employer contribution fields only for relevant account types via Redux state check.
- Configured error boundaries with fallback UI for each Chart.js visualisation widget ensuring individual chart failure does not crash full Moneybox portfolio dashboard during API timeout or malformed response scenarios. • Built Storybook component documentation for 30+ financial UI components (PortfolioCard, HoldingRow, ContributionGauge, PriceSparkline) with Chromatic visual regression CI catching unintended style changes on every PR.
- Achieved Lighthouse performance score of 93 on portfolio dashboard implementing critical CSS inlining, Chart.js dynamic import(), and React.lazy for heavy ISA comparison table component loaded on user interaction. • Wrote 150+ Jest unit tests for Redux Thunk reducers, Chart.js data transformation utilities, and RBAC-controlled dashboard render trees with MSW API mocking achieving 84% coverage enforced in GitHub Actions CI. • Reduced overall dashboard rendering time by 30% through memoised Chart.js data processing pipelines, server side pagination for transaction history, and prefetching portfolio data on route hover using Next.js Link.

PROJECT 5: CareBook – Huma Healthcare Appointment Portal | Domain: Healthcare Client: Huma (UK-based digital health SaaS) | Domain: Healthcare / Patient Scheduling

Tech Stack: Next.js, TypeScript, Bootstrap, JWT, AWS Amplify, GitHub Actions, Jest, React Testing Library

Description: Secure healthcare appointment booking portal enabling 50,000+ patients to search specialists, book teleconsultations, manage medical records, and receive automated appointment reminders via multi-channel notification.

Roles & Responsibilities:

- Developed Next.js SSR pages for doctor discovery and specialty search ensuring patient-facing pages are server rendered for SEO indexability and fast initial load without client-side data fetching waterfall. • Built real-time appointment slot availability grid using React API polling every 60 seconds via useEffect showing live slot status changes without full page reload during active patient booking sessions. • Implemented JWT authentication with MFA enforcement using TOTP via authenticator app requiring second factor for appointment booking preventing unauthorised access to patient health records.
- Developed TypeScript strict interfaces for AppointmentSlot, DoctorProfile, MedicalRecord, and Notification entities enabling compile-time type safety across all booking portal data fetching and rendering workflows. • Configured RBAC with Next.js middleware checking JWT role claims (patient, doctor, admin) at edge layer redirecting unauthenticated requests before Next.js SSR layer processes the request.
- Deployed application on AWS Amplify with CloudFront CDN configuration achieving sub-200ms global content delivery and 35% page load time reduction via edge caching of static appointment page assets. • Implemented

email and SMS appointment reminder integration via REST API: booking confirmation triggers notification dispatch job with rescheduling link, cancellation link, and clinic contact details per notification. • Built secure session management using httpOnly cookie JWT storage with sliding expiry detecting idle timeout and prompting re-authentication preventing unauthorised portal access on shared clinical workstations. • Wrote 140+ Jest and React Testing Library unit tests for booking form validation, slot availability polling, and RBAC middleware with MSW intercepting appointment and doctor profile APIs achieving 82% coverage. • Optimised frontend performance reducing page load time by 35% through Bootstrap CSS tree-shaking via PurgeCSS, React.lazy loading of medical record viewer, and next/image optimisation for doctor profile photos.